Vous : bsr

Gemini a dit

Bonsoir ! Comment puis-je vous aider ce soir ?

Vous avez dit

Avec PyQt6 pouvez vous utiliser python pour créer un logiciel de cryptage et de décryptage de l'importe quel fichier ?

Gemini a dit

Oui, c'est tout à fait possible et c'est un excellent projet. Python excelle dans la manipulation de données et la cryptographie, tandis que PyQt6 est parfait pour créer une interface utilisateur intuitive pour sélectionner les fichiers et lancer les opérations.

Pour réaliser cela, on utilise généralement la bibliothèque cryptography pour gérer la logique de sécurité (le chiffrement et le déchiffrement) et PyQt6 pour l'interface graphique.

Voici comment structurer ce logiciel et un exemple de code fonctionnel pour démarrer.

1. Les prérequis

Il vous faudra installer les deux bibliothèques si ce n'est pas déjà fait :

Bash

```
pip install PyQt6 cryptography
```

2. Le fonctionnement

- **Algorithme** : Nous allons utiliser Fernet de la bibliothèque cryptography. C'est un système de chiffrement symétrique (la même clé sert à chiffrer et à déchiffrer) robuste et standard.
- **Interface (PyQt6)** : Elle utilisera QFileDialog pour permettre à l'utilisateur de parcourir son ordinateur et de choisir n'importe quel fichier (texte, image, PDF, vidéo, etc.), car nous traiterons les fichiers en mode binaire (rb et wb).

3. Exemple de code complet

Ce script crée une interface simple avec un bouton pour générer une clé de sécurité, et des boutons pour choisir un fichier et le chiffrer/déchiffrer.

Python

```python
import sys
import os
from PyQt6.QtWidgets import (QApplication, QWidget, QVBoxLayout, QPushButton,
                             QFileDialog, QLabel, QMessageBox)
from cryptography.fernet import Fernet


class LogicielCryptage(QWidget):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Chiffrement de Fichiers")
        self.resize(400, 200)

        self.chemin_fichier = None
        self.cle = None
        self.fichier_cle = "ma_cle_secrete.key"

        self.initUI()
        self.charger_ou_generer_cle()

    def initUI(self):
```

```
    layout = QVBoxLayout()

    self.label_info = QLabel("Aucun fichier sélectionné")
    layout.addWidget(self.label_info)

    self.btn_choisir = QPushButton("1. Choisir un fichier")
    self.btn_choisir.clicked.connect(self.choisir_fichier)
    layout.addWidget(self.btn_choisir)

    self.btn_chiffrer = QPushButton("2. Chiffrer le fichier")
    self.btn_chiffrer.clicked.connect(self.chiffrer_fichier)
    layout.addWidget(self.btn_chiffrer)

    self.btn_dechiffrer = QPushButton("3. Déchiffrer le fichier")
    self.btn_dechiffrer.clicked.connect(self.dechiffrer_fichier)
    layout.addWidget(self.btn_dechiffrer)

    self.setLayout(layout)

def charger_ou_generer_cle(self):
    """Génère une clé si elle n'existe pas, sinon la charge."""
```

```
    if not os.path.exists(self.fichier_cle):
        self.cle = Fernet.generate_key()
        with open(self.fichier_cle, "wb") as f_cle:
            f_cle.write(self.cle)
    else:
        with open(self.fichier_cle, "rb") as f_cle:
            self.cle = f_cle.read()

def choisir_fichier(self):
    chemin, _ = QFileDialog.getOpenFileName(self, "Sélectionner un fichier")
    if chemin:
        self.chemin_fichier = chemin
        self.label_info.setText(f"Fichier : {os.path.basename(chemin)}")

def chiffrer_fichier(self):
    if not self.chemin_fichier:
        QMessageBox.warning(self, "Erreur", "Veuillez d'abord choisir un fichier.")
        return

    try:
        fernet = Fernet(self.cle)
```

```
        # Lecture des données binaires originales
        with open(self.chemin_fichier, "rb") as f:
            donnees_originales = f.read()

        # Chiffrement
        donnees_chiffrees = fernet.encrypt(donnees_originales)

        # Sauvegarde du nouveau fichier chiffré
        fichier_chiffre = self.chemin_fichier + ".enc"
        with open(fichier_chiffre, "wb") as f:
            f.write(donnees_chiffrees)

        QMessageBox.information(self, "Succès", f"Fichier chiffré avec succès :\n{fichier_chiffre}")
    except Exception as e:
        QMessageBox.critical(self, "Erreur", f"Une erreur est survenue lors du chiffrement :\n{str(e)}")

def dechiffrer_fichier(self):
    if not self.chemin_fichier:
        QMessageBox.warning(self, "Erreur", "Veuillez d'abord choisir un fichier.")
        return
```

```
try:
    fernet = Fernet(self.cle)

    with open(self.chemin_fichier, "rb") as f:
        donnees_chiffrees = f.read()

    # Déchiffrement
    donnees_dechiffrees = fernet.decrypt(donnees_chiffrees)

    # Enlever l'extension .enc si elle est présente pour le fichier de sortie
    if self.chemin_fichier.endswith(".enc"):
        fichier_dechiffre = self.chemin_fichier[:-4]
    else:
        fichier_dechiffre = self.chemin_fichier + ".dec"

    with open(fichier_dechiffre, "wb") as f:
        f.write(donnees_dechiffrees)

    QMessageBox.information(self, "Succès", f"Fichier déchiffré avec succès :\n{fichier_dechiffre}")
except Exception as e:
```

```
        QMessageBox.critical(self, "Erreur", "Clé incorrecte ou fichier corrompu.")


if __name__ == "__main__":
    app = QApplication(sys.argv)
    fenetre = LogicielCryptage()
    fenetre.show()
    sys.exit(app.exec())
```

Points importants à considérer

- **La gestion de la clé** : Dans cet exemple simple, la clé est sauvegardée dans un fichier local ma_cle_secrete.key. C'est elle qui verrouille et déverrouille les fichiers. Si ce fichier .key est perdu, les fichiers chiffrés sont irrécupérables.
- **Les gros fichiers** : Le code ci-dessus lit le fichier entier dans la mémoire vive (RAM) avant de le chiffrer. Pour de très gros fichiers (par exemple une vidéo de 2 Go), il faudrait modifier le code pour lire et chiffrer le fichier par "morceaux" (chunks) afin de ne pas saturer la mémoire.

Voulez-vous que l'on adapte ce code pour qu'il génère la clé à partir d'un mot de passe saisi par l'utilisateur (ce qui est souvent plus pratique qu'un fichier clé séparé) ?

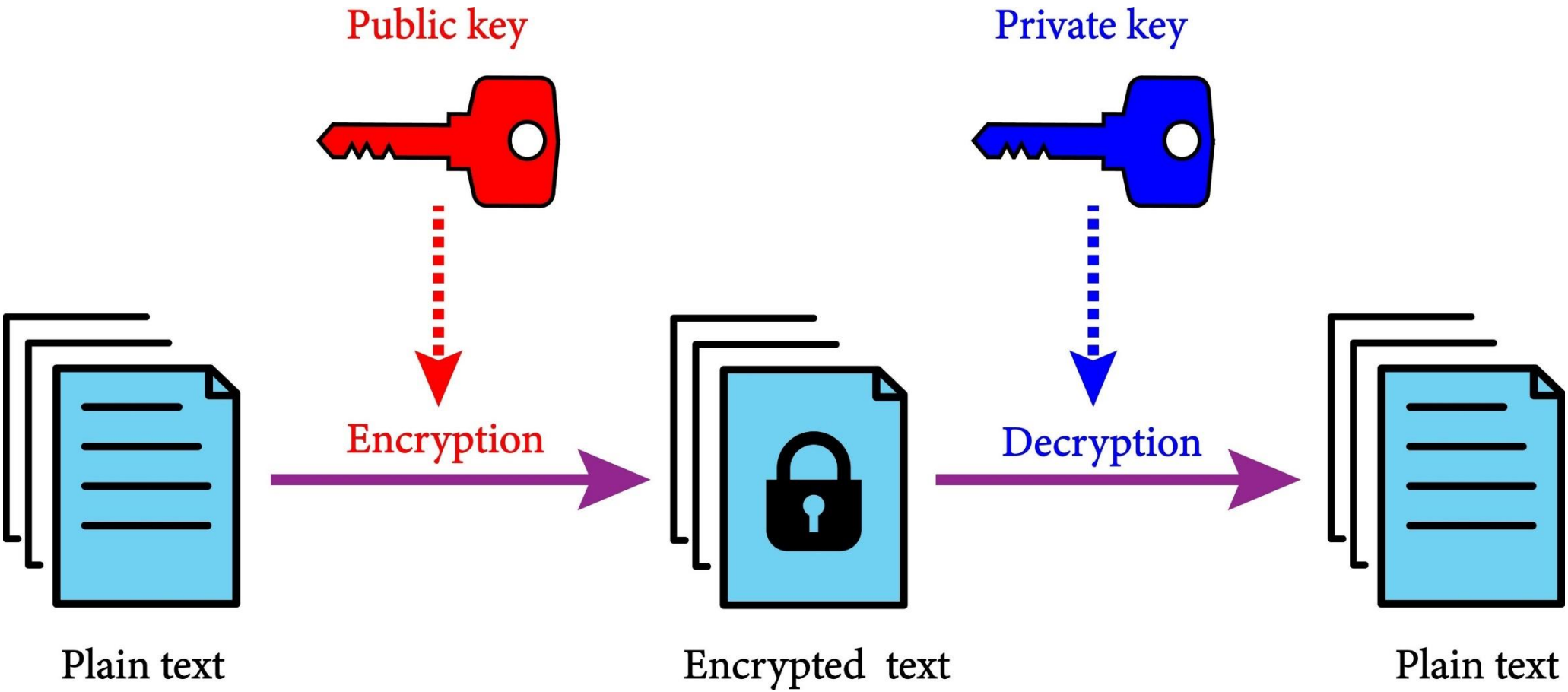
Vous avez dit

Comment appelle t on la methode de cryptographie qe vous avez utilisé ici ?

Gemini a dit

La méthode de cryptographie utilisée dans le code précédent s'appelle le **chiffrement symétrique** (ou cryptographie à clé secrète).

Plus spécifiquement, nous avons utilisé l'implémentation **Fernet** de la bibliothèque Python.



Shutterstock

Voici les détails de cette méthode pour bien comprendre comment elle fonctionne :

1. Le concept du Chiffrement Symétrique

Le principe fondamental du chiffrement symétrique est que **la même clé** est utilisée pour les deux opérations :

- **Chiffrer** (verrouiller) les données d'origine.
- **Déchiffrer** (déverrouiller) les données pour les rendre à nouveau lisibles.

C'est exactement comme le cadenas de votre porte : la clé que vous utilisez en partant pour fermer la porte est la même que celle que vous utilisez en rentrant pour l'ouvrir. C'est une méthode très rapide, ce qui la rend idéale pour chiffrer des fichiers, même volumineux.

2. Ce qu'il y a "sous le capot" de Fernet

Fernet n'est pas un algorithme de base, c'est une "recette" sécurisée prête à l'emploi qui combine plusieurs standards cryptographiques très robustes :

- **Algorithme de chiffrement :** Il utilise **AES** (Advanced Encryption Standard) avec une clé de 128 bits en mode CBC. C'est l'un des standards les plus utilisés et sécurisés au monde (utilisé notamment par les banques et les gouvernements).
- **Authentification des données :** Il utilise **HMAC** (Hash-based Message Authentication Code) avec SHA256. Cela permet de s'assurer que le fichier chiffré n'a pas été modifié ou corrompu par un tiers. Si quelqu'un modifie ne serait-ce qu'un seul octet du fichier chiffré, Fernet refusera de le déchiffrer et lèvera une erreur.

(Note de vocabulaire : En français, le terme exact est "chiffrement" et non "cryptage", même si ce dernier est très courant dans le langage de tous les jours !)

L'alternative au chiffrement symétrique est le **chiffrement asymétrique** (comme le RSA), qui utilise deux clés différentes : une clé *publique* pour chiffrer et une clé *privée* pour déchiffrer.

Souhaitez-vous que nous explorions comment remplacer le fichier de la clé secrète par un mot de passe que l'utilisateur devra taper dans l'interface PyQt6 ?

Vous avez dit

Pouvez vous expliquer comment vous avz généreé le fichier ma_cle_secrete.key

Gemini a dit

Bien sûr ! La création de ce fichier se déroule entièrement dans cette partie spécifique du code, à l'intérieur de la méthode charger_ou_generer_cle :

Python

```
def charger_ou_generer_cle(self):
    """Génère une clé si elle n'existe pas, sinon la charge."""
    if not os.path.exists(self.fichier_cle):
```

```
        # 1. Génération de la clé
        self.cle = Fernet.generate_key()

        # 2. Création et écriture du fichier
        with open(self.fichier_cle, "wb") as f_cle:
            f_cle.write(self.cle)
    else:
        # 3. Lecture si le fichier existe déjà
        with open(self.fichier_cle, "rb") as f_cle:
            self.cle = f_cle.read()
```

Voici l'explication étape par étape de ce qui se passe sous le capot :

1. La vérification de sécurité (os.path.exists**)**

Avant de créer quoi que ce soit, le programme vérifie si le fichier ma_cle_secrete.key existe déjà dans le même dossier que votre script Python.

- C'est une étape cruciale : si le logiciel générait une nouvelle clé à chaque démarrage, **vous perdriez à tout jamais la possibilité de déchiffrer les fichiers** que vous auriez chiffrés la veille !

2. La création mathématique de la clé (Fernet.generate_key()**)**

Si le fichier n'existe pas (par exemple, lors du tout premier lancement de votre logiciel), le programme appelle cette fonction de la bibliothèque cryptography.

- Elle va générer une suite de caractères aléatoire de 32 octets, encodée en URL-safe base64.
- Pour faire simple : elle fabrique une clé numérique unique et extrêmement complexe, impossible à deviner, qui servira de "cadenas" pour vos fichiers.

3. La sauvegarde physique (with open(..., "wb")**)**

Une fois la clé générée en mémoire, il faut la sauvegarder sur votre disque dur pour pouvoir la réutiliser plus tard.

- open(self.fichier_cle, "wb") : Le programme crée le fichier ma_cle_secrete.key et l'ouvre en mode **"wb"** (Write Binary - écriture binaire). On utilise le mode binaire car la clé générée par Fernet est une séquence d'octets purs, pas un texte classique.
- f_cle.write(self.cle) : Il insère la clé à l'intérieur du fichier et le referme proprement.

Une note sur la sécurité : Bien que cette méthode soit parfaite pour comprendre le fonctionnement du chiffrement, stocker sa clé secrète en clair dans un fichier juste à côté du logiciel n'est pas idéal pour la sécurité (si quelqu'un vole votre ordinateur, il a accès au logiciel *et* à la clé).

Souhaitez-vous que je modifie le code PyQt6 pour qu'il n'y ait plus de fichier .key, et qu'à la place, l'utilisateur doive **taper un mot de passe de son choix** dans une boîte de dialogue pour chiffrer ou déchiffrer le fichier ?

Vous avez dit

Est ce que le contenu du fichier ma_cle_secrete.key est t il aléatoire

Gemini a dit

Oui, absolument. Le contenu généré par Fernet.generate_key() et sauvegardé dans ce fichier est **hautement aléatoire**, et c'est précisément ce qui garantit la sécurité de vos fichiers chiffrés.

Voici ce qui se passe techniquement concernant ce caractère aléatoire :

- **Un aléatoire "sécurisé" :** La fonction ne se contente pas d'un aléatoire mathématique simple (qui pourrait être prédit par un ordinateur). Elle fait appel au générateur de nombres aléatoires cryptographiquement sécurisé (CSPRNG) de votre système d'exploitation. Cela crée un aléatoire dit "fort", imprévisible.
- **La puissance de la clé :** Elle génère 32 octets de données pures. Une moitié est utilisée pour le chiffrement (AES-128) et l'autre pour s'assurer que le fichier n'a pas été modifié. Pour deviner la clé de chiffrement au hasard, un ordinateur devrait essayer 2128 combinaisons différentes, ce qui prendrait des milliards d'années avec la technologie actuelle.
- **Le format d'affichage (Base64) :** Même si la clé est une suite de données binaires aléatoires, si vous ouvrez ma_cle_secrete.key avec un éditeur de texte (comme le Bloc-notes), vous ne verrez pas une bouillie de symboles illisibles. La bibliothèque transforme ces octets en texte lisible en utilisant un encodage standard appelé **Base64**.

Le contenu de votre fichier ressemblera toujours à une chaîne de **44 caractères** comprenant des lettres (majuscules et minuscules), des chiffres, et parfois des tirets - ou des tirets du bas _ (et se terminant généralement par un signe =).

Exemple de ce que vous pourriez voir en ouvrant le fichier :

cw_z5Q2a8K1jR-9pL4Xy7vN3mB6fT0dE_hYcWsVqMko=

Le problème de la clé 100% aléatoire

L'inconvénient d'une clé parfaitement aléatoire de 44 caractères, c'est qu'aucun être humain ne peut la retenir de tête. C'est pour cela que notre programme l'a sauvegardée dans un fichier .key posé sur votre disque dur. Mais si votre disque dur lâche et que vous perdez ce fichier, vos données chiffrées seront perdues à jamais.

Souhaitez-vous que je vous montre comment adapter votre interface PyQt6 pour que l'utilisateur puisse utiliser **un mot de passe de son choix** à la place de ce fichier (le programme se chargera alors de transformer mathématiquement ce mot de passe humain en une clé sécurisée) ?

Vous avez dit

Pouvez vous améliorer l'Affichage PyQt6 et rendre responsive ? svp ?

Gemini a dit

C'est une excellente idée. Pour qu'une interface PyQt6 soit à la fois esthétique et responsive (c'est-à-dire qu'elle s'adapte parfaitement quand on redimensionne la fenêtre), il faut abandonner les tailles fixes et utiliser intelligemment les **Layouts** (QVBoxLayout, QHBoxLayout) ainsi que les **feuilles de style (QSS)**, qui fonctionnent de manière très similaire au CSS pour le web.

Voici une version largement améliorée. J'ai ajouté un design moderne avec des coins arrondis, des effets de survol sur les boutons, et une structure qui s'étire et se centre automatiquement.

Le code PyQt6 amélioré et responsive

Vous pouvez remplacer votre ancien code par celui-ci :

Python

```
import sys

import os
```

```
from PyQt6.QtWidgets import (QApplication, QWidget, QVBoxLayout, QHBoxLayout,
                             QPushButton, QFileDialog, QLabel, QMessageBox,
                             QFrame, QSizePolicy, QSpacerItem)
from PyQt6.QtCore import Qt
from PyQt6.QtGui import QFont
from cryptography.fernet import Fernet

class LogicielCryptage(QWidget):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("SecureFile - Chiffrement & Déchiffrement")
        self.resize(500, 350)
        self.setMinimumSize(400, 250) # Empêche de trop réduire la fenêtre

        self.chemin_fichier = None
        self.cle = None
        self.fichier_cle = "ma_cle_secrete.key"

        self.initUI()
        self.appliquer_style()
        self.charger_ou_generer_cle()
```

```
def initUI(self):
    # Layout principal (Vertical)
    layout_principal = QVBoxLayout()
    layout_principal.setContentsMargins(30, 30, 30, 30)
    layout_principal.setSpacing(20)

    # --- Titre ---
    titre = QLabel("Sécurisez vos fichiers")
    titre.setObjectName("titre")
    titre.setAlignment(Qt.AlignmentFlag.AlignCenter)
    layout_principal.addWidget(titre)

    # --- Zone de sélection de fichier (Frame) ---
    frame_fichier = QFrame()
    frame_fichier.setObjectName("frameFichier")
    layout_fichier = QVBoxLayout(frame_fichier)

    self.label_info = QLabel("Aucun fichier sélectionné")
    self.label_info.setAlignment(Qt.AlignmentFlag.AlignCenter)
    self.label_info.setWordWrap(True) # Permet le retour à la ligne si le nom est long
```

```
self.btn_choisir = QPushButton("Parcourir les fichiers...")
self.btn_choisir.setObjectName("btnChoisir")
self.btn_choisir.setCursor(Qt.CursorShape.PointingHandCursor)
self.btn_choisir.clicked.connect(self.choisir_fichier)

layout_fichier.addWidget(self.label_info)
layout_fichier.addWidget(self.btn_choisir, alignment=Qt.AlignmentFlag.AlignCenter)

layout_principal.addWidget(frame_fichier)

# --- Espace flexible ---
layout_principal.addSpacerItem(QSpacerItem(20, 20, QSizePolicy.Policy.Minimum, QSizePolicy.Policy.Expanding))

# --- Boutons d'action (Horizontal) ---
layout_boutons = QHBoxLayout()
layout_boutons.setSpacing(15)

self.btn_chiffrer = QPushButton("🔒 Chiffrer")
self.btn_chiffrer.setObjectName("btnAction")
self.btn_chiffrer.setCursor(Qt.CursorShape.PointingHandCursor)
```

```
    self.btn_chiffrer.clicked.connect(self.chiffrer_fichier)

    self.btn_dechiffrer = QPushButton("🔓 Déchiffrer")
    self.btn_dechiffrer.setObjectName("btnAction")
    self.btn_dechiffrer.setCursor(Qt.CursorShape.PointingHandCursor)
    self.btn_dechiffrer.clicked.connect(self.dechiffrer_fichier)

    # Les boutons prennent la même largeur
    self.btn_chiffrer.setSizePolicy(QSizePolicy.Policy.Expanding, QSizePolicy.Policy.Fixed)
    self.btn_dechiffrer.setSizePolicy(QSizePolicy.Policy.Expanding, QSizePolicy.Policy.Fixed)

    layout_boutons.addWidget(self.btn_chiffrer)
    layout_boutons.addWidget(self.btn_dechiffrer)

    layout_principal.addLayout(layout_boutons)
    self.setLayout(layout_principal)

def appliquer_style(self):
    """Application du QSS (Qt Style Sheets) pour un design moderne"""
    style = """
        QWidget {
```

```
    background-color: #f4f5f7;
    font-family: 'Segoe UI', Arial, sans-serif;
    color: #333333;
}
QLabel#titre {
    font-size: 22px;
    font-weight: bold;
    color: #2c3e50;
    margin-bottom: 10px;
}
QFrame#frameFichier {
    background-color: #ffffff;
    border: 2px dashed #bdc3c7;
    border-radius: 10px;
    padding: 20px;
}
QLabel {
    font-size: 14px;
    color: #7f8c8d;
}
QPushButton {
```

```
    font-size: 14px;
    font-weight: bold;
    padding: 10px 20px;
    border-radius: 6px;
    border: none;
    color: white;
}
QPushButton#btnChoisir {
    background-color: #3498db;
    margin-top: 10px;
}
QPushButton#btnChoisir:hover { background-color: #2980b9; }
QPushButton#btnChoisir:pressed { background-color: #1f618d; }

QPushButton#btnAction {
    background-color: #2ecc71;
    padding: 12px;
    font-size: 15px;
}
QPushButton#btnAction:hover { background-color: #27ae60; }
QPushButton#btnAction:pressed { background-color: #1e8449; }
```

```
        """
        self.setStyleSheet(style)

    # ... (Les méthodes charger_ou_generer_cle, choisir_fichier, chiffrer_fichier et dechiffrer_fichier restent exactement les mêmes que dans le code précédent)
...

    def charger_ou_generer_cle(self):
        if not os.path.exists(self.fichier_cle):
            self.cle = Fernet.generate_key()
            with open(self.fichier_cle, "wb") as f_cle:
                f_cle.write(self.cle)
        else:
            with open(self.fichier_cle, "rb") as f_cle:
                self.cle = f_cle.read()

    def choisir_fichier(self):
        chemin, _ = QFileDialog.getOpenFileName(self, "Sélectionner un fichier")
        if chemin:
            self.chemin_fichier = chemin
            nom_fichier = os.path.basename(chemin)
            self.label_info.setText(f"<b>Fichier prêt :</b> {nom_fichier}")
```

```
        self.label_info.setStyleSheet("color: #2c3e50;")

    def chiffrer_fichier(self):
        if not self.chemin_fichier:
            QMessageBox.warning(self, "Erreur", "Veuillez d'abord choisir un fichier.")
            return
        try:
            fernet = Fernet(self.cle)
            with open(self.chemin_fichier, "rb") as f:
                donnees_originales = f.read()
            donnees_chiffrees = fernet.encrypt(donnees_originales)
            fichier_chiffre = self.chemin_fichier + ".enc"
            with open(fichier_chiffre, "wb") as f:
                f.write(donnees_chiffrees)
            QMessageBox.information(self, "Succès", f"Fichier chiffré :\n{fichier_chiffre}")
        except Exception as e:
            QMessageBox.critical(self, "Erreur", f"Erreur de chiffrement :\n{str(e)}")

    def dechiffrer_fichier(self):
        if not self.chemin_fichier:
            QMessageBox.warning(self, "Erreur", "Veuillez d'abord choisir un fichier.")
```

```
            return
        try:
            fernet = Fernet(self.cle)
            with open(self.chemin_fichier, "rb") as f:
                donnees_chiffrees = f.read()
            donnees_dechiffrees = fernet.decrypt(donnees_chiffrees)
            if self.chemin_fichier.endswith(".enc"):
                fichier_dechiffre = self.chemin_fichier[:-4]
            else:
                fichier_dechiffre = self.chemin_fichier + ".dec"
            with open(fichier_dechiffre, "wb") as f:
                f.write(donnees_dechiffrees)
            QMessageBox.information(self, "Succès", f"Fichier déchiffré :\n{fichier_dechiffre}")
        except Exception as e:
            QMessageBox.critical(self, "Erreur", "Clé incorrecte ou fichier corrompu.")

if __name__ == "__main__":
    app = QApplication(sys.argv)
    fenetre = LogicielCryptage()
    fenetre.show()
    sys.exit(app.exec())
```

Qu'est-ce qui rend cette version responsive et esthétique ?

1. **L'utilisation stricte des Layouts :** Au lieu de dire "place le bouton à 10 pixels du bord", j'ai utilisé QVBoxLayout (Vertical) et QHBoxLayout (Horizontal). Lorsque vous agrandissez la fenêtre, les boîtes s'étirent proportionnellement pour remplir l'espace.
2. QSizePolicy.Policy.Expanding **:** J'ai appliqué cette règle aux boutons "Chiffrer" et "Déchiffrer" pour qu'ils se partagent toujours exactement 50% de la largeur disponible en bas de la fenêtre, quelle que soit sa taille.
3. **Le QSpacerItem :** C'est un "ressort" invisible ajouté au milieu de la fenêtre. Il pousse le cadre de sélection vers le haut et les boutons vers le bas, garantissant que l'interface reste aérée même si la fenêtre est agrandie en plein écran.
4. **L'injection de QSS (Qt Style Sheets) :** La méthode appliquer_style() injecte du code de design directement dans l'application. Elle gère les couleurs de fond douces, les bordures en pointillés (dashed), les polices sans sérif modernes (Segoe UI), et surtout les états interactifs comme :hover (quand la souris survole le bouton) et :pressed (quand on clique dessus).

Voulez-vous que l'on implémente la fonctionnalité permettant d'utiliser un **mot de passe** saisi par l'utilisateur à la place du fichier .key pour verrouiller le tout ?

Vous avez dit

Est ce qu'il est possible de crypter un fichier qui est déjà crypté

Gemini a dit

Oui, c'est tout à fait possible ! En cryptographie, on appelle cela le **sur-chiffrement** (ou chiffrement en cascade).

Pour comprendre pourquoi c'est possible, il faut se rappeler qu'un algorithme de chiffrement (comme Fernet) ne se soucie pas de ce que représente le fichier. Qu'il s'agisse d'une image, d'un texte clair, ou d'une bouillie d'octets déjà chiffrée, pour l'algorithme, ce ne sont que des données binaires (des 0 et des 1). Il prendra ces données et les mélangera à nouveau.

C'est exactement le principe des poupées russes, ou le fait de mettre un coffre-fort verrouillé à l'intérieur d'un autre coffre-fort verrouillé.

Si vous utilisez votre logiciel PyQt6 actuel et que vous cliquez deux fois de suite sur "Chiffrer" pour le même fichier, voici ce qui va se passer concrètement :

1. Le nom du fichier

Le programme va ajouter l'extension à chaque fois. Votre fichier document.pdf deviendra document.pdf.enc, et si vous le rechiffrez, il deviendra document.pdf.enc.enc.

2. La taille du fichier

À chaque couche de chiffrement, **le fichier va légèrement grossir**. Fernet n'ajoute pas seulement le texte chiffré, il ajoute aussi des métadonnées essentielles pour la sécurité (un horodatage, un vecteur d'initialisation pour l'aléatoire, et la fameuse signature HMAC pour vérifier l'intégrité). Ces informations supplémentaires pèsent quelques dizaines d'octets à chaque fois.

3. La logique de déchiffrement

Pour retrouver votre fichier d'origine, vous devrez obligatoirement faire le chemin inverse dans l'ordre exact. Si vous avez chiffré le fichier 3 fois, vous devrez cliquer 3 fois sur "Déchiffrer".

- Si vous avez utilisé des clés différentes pour chaque couche, il faudra utiliser les clés dans l'ordre inverse du chiffrement (c'est d'ailleurs ce principe "en couches d'oignon" qui sécurise le célèbre réseau Tor).
- Si vous avez utilisé la même clé (comme c'est le cas dans votre logiciel actuel), la même clé servira à déverrouiller toutes les couches une par une.

Est-ce que cela rend le fichier plus sécurisé ?

- **Avec la même clé : Non.** Chiffrer deux fois avec la même clé n'apporte pratiquement aucune sécurité supplémentaire. C'est comme mettre deux cadenas identiques sur une porte : si le voleur a la clé pour le premier, il a la clé pour le second.
- **Avec des clés différentes : Oui.** Cela multiplie la difficulté pour un attaquant, car il devra casser chaque clé indépendamment.

Maintenant que le fonctionnement de la clé et du chiffrement n'a plus de secret pour vous, voulez-vous que nous ajoutions une fenêtre pop-up dans votre interface PyQt6 pour que l'utilisateur puisse **créer sa propre clé à partir d'un mot de passe** au lieu d'utiliser le fichier .key automatique ?

Vous avez dit

Est ce que l'AES 128 bits est t il le meilleur ? C'est - à -dire est ce qu'il existe un autre algorithme beaucoup plus performant et beaucoup plus efficace que l'AES 128 bits

Gemini a dit

C'est une excellente question. La réponse courte est : **non, AES-128 n'est pas le "meilleur" dans l'absolu**, bien qu'il soit un standard mondial incroyablement robuste et largement suffisant pour 99 % des usages quotidiens.

En cryptographie, le "meilleur" algorithme est toujours une question de compromis entre **la sécurité maximale** et **la performance** (vitesse et consommation d'énergie).

Voici les algorithmes qui surpassent AES-128, soit en sécurité, soit en efficacité :

1. Le grand frère pour une sécurité maximale : AES-256

Si vous cherchez un algorithme plus sécurisé que l'AES-128, son évolution directe est l'**AES-256**.

- **La différence :** La taille de la clé passe de 128 bits à 256 bits. Pour craquer une clé AES-128, il faut tester 2128 combinaisons. Pour AES-256, c'est 2256 (un nombre si gigantesque qu'il dépasse le nombre d'atomes dans l'univers observable).
- **Le standard ultime :** C'est l'algorithme exigé par le gouvernement américain (NSA) pour chiffrer les documents classés "Top Secret". Il est également considéré comme résistant aux futures attaques des ordinateurs quantiques.
- **Le compromis :** Il demande environ 40 % de calculs mathématiques en plus que l'AES-128. Il est donc un tout petit peu plus lent et consomme plus de batterie sur un ordinateur portable ou un smartphone.

2. Le challenger moderne, plus performant : ChaCha20

Si vous cherchez un algorithme plus *efficace* et souvent plus *rapide* que l'AES, le grand gagnant actuel s'appelle **ChaCha20** (souvent associé à Poly1305 pour l'authentification).

- **L'efficacité logicielle :** L'AES a été conçu dans les années 90. Pour être très rapide aujourd'hui, il s'appuie sur des puces physiques dédiées à l'intérieur de nos processeurs (l'accélération matérielle AES-NI). Si un appareil n'a pas cette puce (comme beaucoup de smartphones d'entrée de gamme, d'objets connectés ou d'anciennes tablettes), l'AES devient très lent.
- **La force de ChaCha20 :** Créé beaucoup plus récemment, il est conçu pour être extrêmement rapide **uniquement de manière logicielle**, sans avoir besoin de puce dédiée. Sur un appareil sans accélération matérielle, ChaCha20 est **jusqu'à 3 fois plus rapide** que l'AES, tout en offrant une sécurité équivalente à l'AES-256.
- **Qui l'utilise ?** Google l'utilise massivement pour sécuriser les connexions sur les téléphones Android, Cloudflare l'utilise pour le trafic web, et c'est l'algorithme au cœur du très rapide VPN "WireGuard".

En résumé : lequel choisir ?

- **AES-128 (Fernet) :** Rapide, standard, parfait pour des petits logiciels ou des scripts rapides sur PC.
- **AES-256 :** Pour la paranoïa absolue ou les données industrielles/gouvernementales critiques, sur des ordinateurs puissants.

- **ChaCha20 :** Le meilleur choix moderne pour allier sécurité maximale et vitesse foudroyante, particulièrement si votre logiciel doit tourner sur des appareils moins puissants.

Dans la bibliothèque Python cryptography que nous utilisons, Fernet utilise par défaut AES-128, mais la bibliothèque propose aussi ChaCha20 !

Souhaitez-vous que nous fassions évoluer notre logiciel PyQt6 pour y intégrer **ChaCha20** afin de le rendre ultra-moderne, ou préférez-vous d'abord que nous remplacions le fichier de clé par la **saisie d'un mot de passe** utilisateur ?

Vous avez dit

Pouvez vous faire évoluer notre logiciel PyQt6 pour y intégrer ChaCha20 afin de le rendre ultra-moderne

Gemini a dit

C'est un excellent choix ! Passer à **ChaCha20-Poly1305** rend votre application digne des standards de sécurité les plus récents, tout en garantissant des performances optimales.

Pour faire cette évolution, il y a une petite différence technique importante à comprendre par rapport à Fernet. ChaCha20 a besoin d'un élément supplémentaire appelé un **Nonce** (Number Used Once).

- **Qu'est-ce que le Nonce ?** C'est une petite valeur aléatoire de 12 octets qui doit être générée *à chaque fois* que vous chiffrez un fichier.
- **Comment on le gère ?** Pour pouvoir déchiffrer le fichier plus tard, le programme a besoin de ce Nonce. L'astuce standard en cryptographie consiste simplement à "coller" ces 12 octets tout au début de votre fichier chiffré. Lors du déchiffrement, le programme lira les 12 premiers octets pour récupérer le Nonce, puis déchiffrera le reste du fichier.

Voici le code complet et mis à jour de votre application PyQt6, intégrant ChaCha20-Poly1305 tout en conservant le design moderne et responsive :

Le code avec ChaCha20-Poly1305

Python

```
import sys

import os

from PyQt6.QtWidgets import (QApplication, QWidget, QVBoxLayout, QHBoxLayout,

                             QPushButton, QFileDialog, QLabel, QMessageBox,
```

```
                    QFrame, QSizePolicy, QSpacerItem)
from PyQt6.QtCore import Qt
from cryptography.hazmat.primitives.ciphers.aead import ChaCha20Poly1305


class LogicielCryptage(QWidget):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("SecureFile - ChaCha20 Edition")
        self.resize(500, 350)
        self.setMinimumSize(400, 250)

        self.chemin_fichier = None
        self.cle = None
        self.fichier_cle = "ma_cle_chacha20.key"

        self.initUI()
        self.appliquer_style()
        self.charger_ou_generer_cle()

    def initUI(self):
        # Layout principal
```

```
layout_principal = QVBoxLayout()
layout_principal.setContentsMargins(30, 30, 30, 30)
layout_principal.setSpacing(20)

titre = QLabel("Sécurité Ultra-Moderne (ChaCha20)")
titre.setObjectName("titre")
titre.setAlignment(Qt.AlignmentFlag.AlignCenter)
layout_principal.addWidget(titre)

# Zone de sélection
frame_fichier = QFrame()
frame_fichier.setObjectName("frameFichier")
layout_fichier = QVBoxLayout(frame_fichier)

self.label_info = QLabel("Aucun fichier sélectionné")
self.label_info.setAlignment(Qt.AlignmentFlag.AlignCenter)
self.label_info.setWordWrap(True)

self.btn_choisir = QPushButton("Parcourir les fichiers...")
self.btn_choisir.setObjectName("btnChoisir")
self.btn_choisir.setCursor(Qt.CursorShape.PointingHandCursor)
```

```
self.btn_choisir.clicked.connect(self.choisir_fichier)

layout_fichier.addWidget(self.label_info)
layout_fichier.addWidget(self.btn_choisir, alignment=Qt.AlignmentFlag.AlignCenter)
layout_principal.addWidget(frame_fichier)

layout_principal.addSpacerItem(QSpacerItem(20, 20, QSizePolicy.Policy.Minimum, QSizePolicy.Policy.Expanding))

# Boutons d'action
layout_boutons = QHBoxLayout()
layout_boutons.setSpacing(15)

self.btn_chiffrer = QPushButton("🔒 Chiffrer")
self.btn_chiffrer.setObjectName("btnAction")
self.btn_chiffrer.setCursor(Qt.CursorShape.PointingHandCursor)
self.btn_chiffrer.clicked.connect(self.chiffrer_fichier)

self.btn_dechiffrer = QPushButton("🔓 Déchiffrer")
self.btn_dechiffrer.setObjectName("btnAction")
self.btn_dechiffrer.setCursor(Qt.CursorShape.PointingHandCursor)
self.btn_dechiffrer.clicked.connect(self.dechiffrer_fichier)
```

```
    self.btn_chiffrer.setSizePolicy(QSizePolicy.Policy.Expanding, QSizePolicy.Policy.Fixed)
    self.btn_dechiffrer.setSizePolicy(QSizePolicy.Policy.Expanding, QSizePolicy.Policy.Fixed)

    layout_boutons.addWidget(self.btn_chiffrer)
    layout_boutons.addWidget(self.btn_dechiffrer)

    layout_principal.addLayout(layout_boutons)
    self.setLayout(layout_principal)

def appliquer_style(self):
    style = """
        QWidget { background-color: #f4f5f7; font-family: 'Segoe UI', Arial, sans-serif; color: #333333; }
        QLabel#titre { font-size: 20px; font-weight: bold; color: #8e44ad; margin-bottom: 10px; }
        QFrame#frameFichier { background-color: #ffffff; border: 2px dashed #bdc3c7; border-radius: 10px; padding: 20px; }
        QLabel { font-size: 14px; color: #7f8c8d; }
        QPushButton { font-size: 14px; font-weight: bold; padding: 10px 20px; border-radius: 6px; border: none; color: white; }
        QPushButton#btnChoisir { background-color: #9b59b6; margin-top: 10px; }
        QPushButton#btnChoisir:hover { background-color: #8e44ad; }
        QPushButton#btnChoisir:pressed { background-color: #732d91; }
        QPushButton#btnAction { background-color: #2ecc71; padding: 12px; font-size: 15px; }
```

```
        QPushButton#btnAction:hover { background-color: #27ae60; }
        QPushButton#btnAction:pressed { background-color: #1e8449; }
    """
    self.setStyleSheet(style)

def charger_ou_generer_cle(self):
    """Génère une clé ChaCha20 de 256 bits (32 octets)"""
    if not os.path.exists(self.fichier_cle):
        self.cle = ChaCha20Poly1305.generate_key()
        with open(self.fichier_cle, "wb") as f_cle:
            f_cle.write(self.cle)
    else:
        with open(self.fichier_cle, "rb") as f_cle:
            self.cle = f_cle.read()

def choisir_fichier(self):
    chemin, _ = QFileDialog.getOpenFileName(self, "Sélectionner un fichier")
    if chemin:
        self.chemin_fichier = chemin
        nom_fichier = os.path.basename(chemin)
        self.label_info.setText(f"<b>Fichier prêt :</b> {nom_fichier}")
```

```
        self.label_info.setStyleSheet("color: #2c3e50;")

    def chiffrer_fichier(self):
        if not self.chemin_fichier:
            QMessageBox.warning(self, "Erreur", "Veuillez d'abord choisir un fichier.")
            return
        try:
            chacha = ChaCha20Poly1305(self.cle)
            # Génération d'un Nonce aléatoire de 12 octets
            nonce = os.urandom(12)

            with open(self.chemin_fichier, "rb") as f:
                donnees_originales = f.read()

            # Chiffrement (les données additionnelles 'None' ne sont pas utilisées ici)
            donnees_chiffrees = chacha.encrypt(nonce, donnees_originales, None)

            fichier_chiffre = self.chemin_fichier + ".enc"
            with open(fichier_chiffre, "wb") as f:
                # On sauvegarde le Nonce suivi des données chiffrées
                f.write(nonce + donnees_chiffrees)
```

```
        QMessageBox.information(self, "Succès", f"Fichier chiffré avec ChaCha20 :\n{fichier_chiffre}")
    except Exception as e:
        QMessageBox.critical(self, "Erreur", f"Erreur de chiffrement :\n{str(e)}")

def dechiffrer_fichier(self):
    if not self.chemin_fichier:
        QMessageBox.warning(self, "Erreur", "Veuillez d'abord choisir un fichier.")
        return
    try:
        chacha = ChaCha20Poly1305(self.cle)

        with open(self.chemin_fichier, "rb") as f:
            contenu_complet = f.read()

        # Extraction du Nonce (les 12 premiers octets) et du message chiffré (le reste)
        nonce = contenu_complet[:12]
        donnees_chiffrees = contenu_complet[12:]

        # Déchiffrement
        donnees_dechiffrees = chacha.decrypt(nonce, donnees_chiffrees, None)
```

```
        if self.chemin_fichier.endswith(".enc"):
            fichier_dechiffre = self.chemin_fichier[:-4]
        else:
            fichier_dechiffre = self.chemin_fichier + ".dec"

        with open(fichier_dechiffre, "wb") as f:
            f.write(donnees_dechiffrees)

        QMessageBox.information(self, "Succès", f"Fichier déchiffré :\n{fichier_dechiffre}")
    except Exception as e:
        QMessageBox.critical(self, "Erreur", "Clé incorrecte, fichier corrompu ou Nonce invalide.")

if __name__ == "__main__":
    app = QApplication(sys.argv)
    fenetre = LogicielCryptage()
    fenetre.show()
    sys.exit(app.exec())
```

Ce qui a changé :

1. **L'importation :** Nous sommes passés de Fernet à ChaCha20Poly1305.
2. **Le chiffrement :** La fonction os.urandom(12) crée un Nonce unique. Lors de la sauvegarde, on écrit nonce + donnees_chiffrees dans le fichier .enc.

3. **Le déchiffrement :** Le programme lit le fichier, sépare les 12 premiers octets (contenu_complet[:12]) pour retrouver le Nonce, et utilise le reste (contenu_complet[12:]) pour le déchiffrement.
4. **Le style :** J'ai légèrement adapté les couleurs (tons violets) pour marquer visuellement le passage à cette nouvelle version "Ultra-Moderne".

Maintenant que vous avez un moteur cryptographique de pointe, voulez-vous que nous passions à l'étape finale : ajouter une boîte de dialogue personnalisée pour que l'utilisateur **crée sa clé à partir d'un mot de passe** au lieu du fichier automatique .key ?

Vous avez dit

Pouvez vous afficher le contenu du fichier requirements.txt correspondant ?

Gemini a dit

Voici le contenu exact que vous devez mettre dans votre fichier requirements.txt pour ce projet :

Plaintext

PyQt6

cryptography

Explication rapide des dépendances :

- **PyQt6** : Gère l'intégralité de l'interface graphique (la fenêtre, les boutons, le style responsive, les boîtes de dialogue).
- **cryptography** : Fournit le moteur de sécurité ultra-moderne, incluant l'algorithme ChaCha20-Poly1305 et la gestion des données aléatoires pour le Nonce.

(Note : Les modules os et sys que nous utilisons dans le code sont intégrés nativement à Python, ils n'ont donc pas besoin d'être installés ni listés ici).

Comment l'utiliser ?

Une fois ce fichier requirements.txt créé et placé dans le même dossier que votre script Python, il vous suffit d'ouvrir votre terminal (ou l'invite de commande) dans ce dossier et de taper :

Bash

pip install -r requirements.txt

Cela téléchargera et installera automatiquement les bonnes versions des bibliothèques.

Souhaitez-vous que nous passions à la dernière étape logique de notre logiciel, à savoir la création d'une fenêtre demandant un **mot de passe** à l'utilisateur pour générer la clé, afin de supprimer le fichier .key ?

Vous avez dit

Et maintenant est ce que vous pouvez rendre ce logiciel capable de crypter un dossier contenant beaucoup de sous dossiers et beaucoup de fichiers

Gemini a dit

Oui, c'est une excellente évolution ! C'est ce qui transforme un simple script en un véritable outil de productivité.

Pour réaliser cela, nous allons utiliser une fonction native de Python redoutablement efficace : os.walk().

Cette fonction agit comme un explorateur : elle entre dans le dossier que vous avez choisi, liste tous les fichiers, puis entre dans chaque sous-dossier, liste les fichiers, et ainsi de suite jusqu'à avoir visité l'intégralité de l'arborescence.

Voici le code mis à jour. J'ai modifié le bouton pour qu'il sélectionne un dossier, et j'ai adapté les fonctions de chiffrement pour qu'elles traitent les fichiers par lots.

Le code pour chiffrer/déchiffrer des dossiers complets

Python

```
import sys

import os

from PyQt6.QtWidgets import (QApplication, QWidget, QVBoxLayout, QHBoxLayout,

                    QPushButton, QFileDialog, QLabel, QMessageBox,

                    QFrame, QSizePolicy, QSpacerItem)

from PyQt6.QtCore import Qt

from cryptography.hazmat.primitives.ciphers.aead import ChaCha20Poly1305
```

```
class LogicielCryptage(QWidget):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("SecureFolder - ChaCha20 Edition")
        self.resize(500, 350)
        self.setMinimumSize(400, 250)

        self.chemin_cible = None # Peut être un dossier
        self.cle = None
        self.fichier_cle = "ma_cle_chacha20.key"

        self.initUI()
        self.appliquer_style()
        self.charger_ou_generer_cle()

    def initUI(self):
        layout_principal = QVBoxLayout()
        layout_principal.setContentsMargins(30, 30, 30, 30)
        layout_principal.setSpacing(20)

        titre = QLabel("Sécurisez vos Dossiers")
```

```
titre.setObjectName("titre")
titre.setAlignment(Qt.AlignmentFlag.AlignCenter)
layout_principal.addWidget(titre)

frame_fichier = QFrame()
frame_fichier.setObjectName("frameFichier")
layout_fichier = QVBoxLayout(frame_fichier)

self.label_info = QLabel("Aucun dossier sélectionné")
self.label_info.setAlignment(Qt.AlignmentFlag.AlignCenter)
self.label_info.setWordWrap(True)

# Modification : Le bouton sélectionne maintenant un dossier
self.btn_choisir = QPushButton("Parcourir les dossiers...")
self.btn_choisir.setObjectName("btnChoisir")
self.btn_choisir.setCursor(Qt.CursorShape.PointingHandCursor)
self.btn_choisir.clicked.connect(self.choisir_dossier)

layout_fichier.addWidget(self.label_info)
layout_fichier.addWidget(self.btn_choisir, alignment=Qt.AlignmentFlag.AlignCenter)
layout_principal.addWidget(frame_fichier)
```

```
layout_principal.addSpacerItem(QSpacerItem(20, 20, QSizePolicy.Policy.Minimum, QSizePolicy.Policy.Expanding))

layout_boutons = QHBoxLayout()
layout_boutons.setSpacing(15)

self.btn_chiffrer = QPushButton("🔒 Chiffrer le dossier")
self.btn_chiffrer.setObjectName("btnAction")
self.btn_chiffrer.clicked.connect(self.chiffrer_dossier)

self.btn_dechiffrer = QPushButton("🔓 Déchiffrer le dossier")
self.btn_dechiffrer.setObjectName("btnAction")
self.btn_dechiffrer.clicked.connect(self.dechiffrer_dossier)

self.btn_chiffrer.setSizePolicy(QSizePolicy.Policy.Expanding, QSizePolicy.Policy.Fixed)
self.btn_dechiffrer.setSizePolicy(QSizePolicy.Policy.Expanding, QSizePolicy.Policy.Fixed)

layout_boutons.addWidget(self.btn_chiffrer)
layout_boutons.addWidget(self.btn_dechiffrer)

layout_principal.addLayout(layout_boutons)
```

```
    self.setLayout(layout_principal)

def appliquer_style(self):
    style = """
        QWidget { background-color: #f4f5f7; font-family: 'Segoe UI', Arial, sans-serif; color: #333333; }
        QLabel#titre { font-size: 20px; font-weight: bold; color: #e67e22; margin-bottom: 10px; }
        QFrame#frameFichier { background-color: #ffffff; border: 2px dashed #bdc3c7; border-radius: 10px; padding: 20px; }
        QLabel { font-size: 14px; color: #7f8c8d; }
        QPushButton { font-size: 14px; font-weight: bold; padding: 10px 20px; border-radius: 6px; border: none; color: white; }
        QPushButton#btnChoisir { background-color: #d35400; margin-top: 10px; }
        QPushButton#btnChoisir:hover { background-color: #e67e22; }
        QPushButton#btnChoisir:pressed { background-color: #ba4a00; }
        QPushButton#btnAction { background-color: #2ecc71; padding: 12px; font-size: 15px; }
        QPushButton#btnAction:hover { background-color: #27ae60; }
        QPushButton#btnAction:pressed { background-color: #1e8449; }
    """
    self.setStyleSheet(style)

def charger_ou_generer_cle(self):
    if not os.path.exists(self.fichier_cle):
        self.cle = ChaCha20Poly1305.generate_key()
```

```
        with open(self.fichier_cle, "wb") as f_cle:
            f_cle.write(self.cle)
    else:
        with open(self.fichier_cle, "rb") as f_cle:
            self.cle = f_cle.read()

def choisir_dossier(self):
    # Modification : getExistingDirectory au lieu de getOpenFileName
    chemin = QFileDialog.getExistingDirectory(self, "Sélectionner un dossier")
    if chemin:
        self.chemin_cible = chemin
        nom_dossier = os.path.basename(chemin)
        self.label_info.setText(f"<b>Dossier prêt :</b> {nom_dossier}")
        self.label_info.setStyleSheet("color: #2c3e50;")

def chiffrer_dossier(self):
    if not self.chemin_cible:
        QMessageBox.warning(self, "Erreur", "Veuillez d'abord choisir un dossier.")
        return

    chacha = ChaCha20Poly1305(self.cle)
```

```
fichiers_traites = 0
erreurs = 0

# Parcours de toute l'arborescence du dossier
for racine, sous_dossiers, fichiers in os.walk(self.chemin_cible):
    for nom_fichier in fichiers:
        # On ignore les fichiers déjà chiffrés pour éviter le double chiffrement
        if nom_fichier.endswith(".enc"):
            continue

        chemin_complet = os.path.join(racine, nom_fichier)

        try:
            nonce = os.urandom(12)
            with open(chemin_complet, "rb") as f:
                donnees = f.read()

            donnees_chiffrees = chacha.encrypt(nonce, donnees, None)

            with open(chemin_complet + ".enc", "wb") as f:
                f.write(nonce + donnees_chiffrees)
```

```
            # IMPORTANT : On supprime le fichier original en clair
            os.remove(chemin_complet)
            fichiers_traites += 1

            # Permet à l'interface de ne pas bloquer complètement
            QApplication.processEvents()

        except Exception:
            erreurs += 1

    QMessageBox.information(self, "Terminé", f"Opération terminée.\n{fichiers_traites} fichiers chiffrés.\n{erreurs} erreurs rencontrées.")

def dechiffrer_dossier(self):
    if not self.chemin_cible:
        QMessageBox.warning(self, "Erreur", "Veuillez d'abord choisir un dossier.")
        return

    chacha = ChaCha20Poly1305(self.cle)
    fichiers_traites = 0
    erreurs = 0
```

```
for racine, sous_dossiers, fichiers in os.walk(self.chemin_cible):
    for nom_fichier in fichiers:
        # On ne traite QUE les fichiers chiffrés
        if not nom_fichier.endswith(".enc"):
            continue

        chemin_complet = os.path.join(racine, nom_fichier)

        try:
            with open(chemin_complet, "rb") as f:
                contenu = f.read()

            nonce = contenu[:12]
            donnees_chiffrees = contenu[12:]
            donnees_dechiffrees = chacha.decrypt(nonce, donnees_chiffrees, None)

            # On retire l'extension .enc
            chemin_original = chemin_complet[:-4]

            with open(chemin_original, "wb") as f:
```

```
                f.write(donnees_dechiffrees)

            # On supprime le fichier chiffré
            os.remove(chemin_complet)
            fichiers_traites += 1

            QApplication.processEvents()

        except Exception:
            erreurs += 1

    QMessageBox.information(self, "Terminé", f"Opération terminée.\n{fichiers_traites} fichiers déchiffrés.\n{erreurs} erreurs rencontrées.")

if __name__ == "__main__":
    app = QApplication(sys.argv)
    fenetre = LogicielCryptage()
    fenetre.show()
    sys.exit(app.exec())
```

Les changements majeurs dans cette version :

1. **Exploration des dossiers :** Le bouton utilise désormais QFileDialog.getExistingDirectory. L'utilisateur sélectionne un dossier global, et le logiciel s'occupe du reste.

2. **Filtrage intelligent :** Lors du chiffrement, le programme ignore les fichiers qui se terminent par .enc. Lors du déchiffrement, il ignore tout ce qui *ne se termine pas* par .enc. Cela évite de casser vos fichiers.
3. **Suppression des originaux (**os.remove**) :** C'est une règle de sécurité vitale. Si vous chiffrez un dossier pour le sécuriser, l'ancienne version en "texte clair" doit disparaître. Le script chiffre le fichier, sauvegarde le .enc, puis détruit l'original. L'inverse se produit au déchiffrement.
4. **Gestion des erreurs :** J'ai encapsulé la lecture/écriture dans un bloc try...except individuel pour chaque fichier. Si un fichier est verrouillé par un autre programme (comme un fichier système), le logiciel comptera une erreur mais continuera à chiffrer les milliers d'autres fichiers au lieu de planter.
5. QApplication.processEvents() **:** Traiter un dossier contenant 5000 fichiers peut prendre plusieurs secondes. Cette petite ligne indique à PyQt6 de "respirer" entre chaque fichier, ce qui empêche votre fenêtre de planter ou d'afficher "(Ne répond pas)" sous Windows.

Voulez-vous que l'on s'attaque maintenant au système de mot de passe utilisateur pour générer la clé, ce qui est particulièrement pertinent si vous chiffrez des dossiers entiers de projets Python ou Django ?

Gemini est une IA et peut se tromper.